

Физически-информированные нейронные сети

Александров Михаил

Уравнения в частных производных

PDE — уравнение на функцию $u(x, t)$, связывающее её производные:

$$F(x, t, u, \nabla u, u_t, \nabla^2 u, \dots) = 0, \quad u : \Omega \times (0, T) \rightarrow \mathbb{R}^m.$$

Классический рецепт — дискретизация области и сведение PDE к системе алгебраических уравнений: конечные разности (FDM), конечные объёмы (FVM), конечные элементы (FEM).

Например, явная схема для уравнения теплопроводности $u_t = u_{xx}$:

$$v_j^{n+1} = (1 - 2\lambda) v_j^n + \lambda(v_{j-1}^n + v_{j+1}^n), \quad \lambda = \frac{\Delta t}{\Delta x^2}.$$

Где классические методы перестают работать

- **Проклятие размерности.** Классический метод — решение большой системы линейных уравнений. Число неизвестных растёт как N^d , где d — размерность задачи, N — размер сетки.
- **Сложные геометрии, multiscale / multiphysics.** Генерация качественной сетки - нетривиальная задача; разные физические масштабы плохо уживаются на одной сетке.
- **Many-query задачи.** Например поиск оптимальный дизайна автомобиля.
- **Обратные задачи.** Восстановить коэффициенты уравнения по данным.
- **Неполная физика.** Есть экспериментальные данные, но полного уравнения нет.

Идея PINN: пример уравнения Бюргерса

Уравнение Бюргерса:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} - \nu \frac{\partial^2 u}{\partial x^2} = 0, \quad x \in [-1, 1], \quad t \in [0, 1]$$

с начальным и граничными условиями: $u(x, 0) = -\sin(\pi x)$, $u(\pm 1, t) = 0$.

Ключевая идея: параметризуем
решение нейросетью

Функция потерь — сумма квадратов
невязок:

$$u_\theta(x, t) \approx u(x, t)$$

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{PDE}}(\theta) + \mathcal{L}_{\text{IC}}(\theta) + \mathcal{L}_{\text{BC}}(\theta),$$

и подбираем параметры θ так,
чтобы сеть удовлетворяла всем
компонентам задачи
одновременно.

$$(x, t) \rightarrow \text{MLP} \rightarrow u_\theta$$

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_r} \sum_n (\partial_t u_\theta + u_\theta \partial_x u_\theta - \nu \partial_{xx} u_\theta)_{(x_n, t_n)}^2$$

$$\mathcal{L}_{\text{IC}} = \frac{1}{N_0} \sum_n (u_\theta(x_n, 0) + \sin(\pi x_n))^2$$

$$\mathcal{L}_{\text{BC}} = \frac{1}{N_b} \sum_n (u_\theta(\pm 1, t_n))^2$$

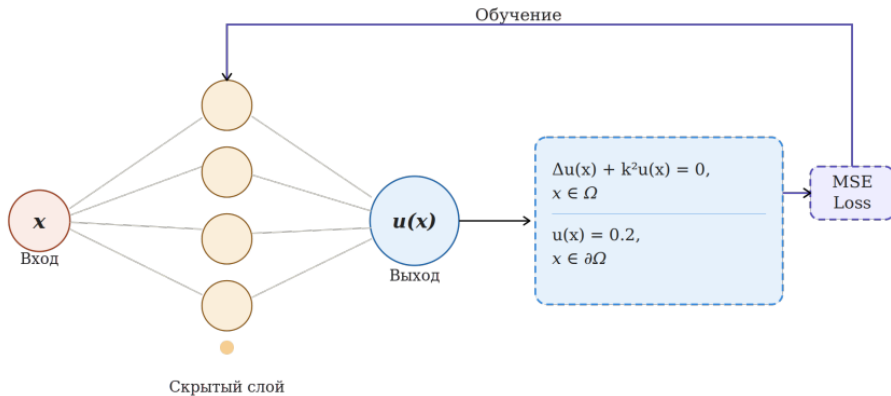
Идея PINN: общая формулировка

Для произвольной краевой задачи $\mathcal{R}_i[u] = f_i$, $\mathcal{B}_j[u] = g_j$:

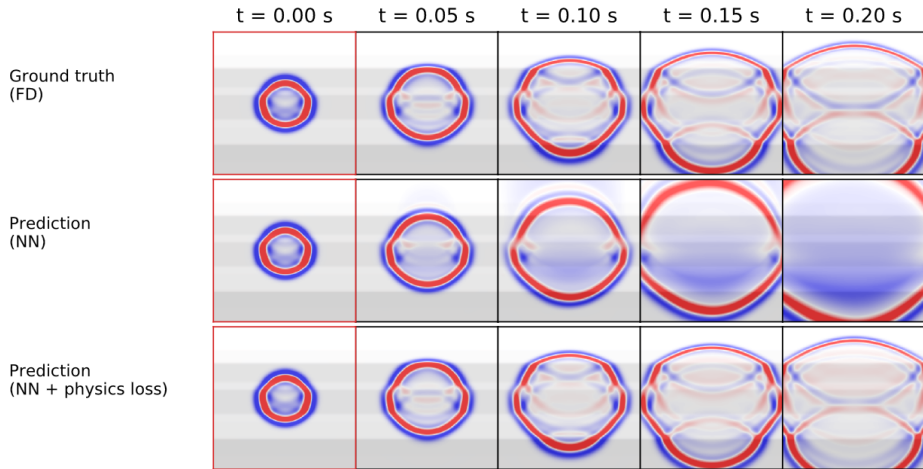
$$\min_{\theta \in \mathbb{R}^d} \mathcal{L}(\theta) = \sum_{i=1}^{M_r} \mathcal{L}_{r,i}(\theta) + \sum_{j=M_r+1}^M \mathcal{L}_{b,j}(\theta),$$

$$\mathcal{L}_{r,i}(\theta) = \frac{1}{N_r} \sum_{n=1}^{N_r} |\mathcal{R}_i[u_\theta](x_r^n) - f_i(x_r^n)|^2, \quad \mathcal{L}_{b,j}(\theta) = \frac{1}{N_b} \sum_{n=1}^{N_b} |\mathcal{B}_j[u_\theta](x_b^n) - g_j(x_b^n)|^2.$$

- Производные можно считать точно (autograd) или численно (конечные разности)
- **Mesh-free.** Коллокационные точки $\{x_r^n\}$ семплируются случайно



Сравнение NN и PINN

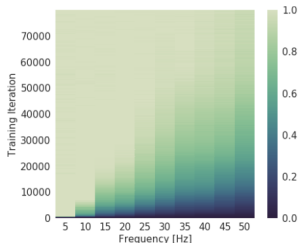


Moseley et al.: Solving the wave equation with physics-informed deep learning

Проблема: Спектральное смещение (Spectral Bias)

Суть проблемы: нейросети отдают приоритет изучению **низкочастотных функций** на ранних этапах.

Подтверждается теорией Neural Tangent Kernel (NTK). Высокие частоты требуют десятков тысяч итераций обучения.



Rahaman et al.: On the Spectral Bias of Neural Networks

Следствие для PINN:

PINN испытывают трудности при решении задач с высокими частотами и многомасштабными явлениями (напр. ударные волны).

При наличии высоких частот:

- Требуется больше коллокационных точек.
- Нужна более крупная модель.
- Спектральное смещение сильно замедляет сходимость.

Решение: Fourier Features

Идея: преобразовать вход в высокочастотные функции.

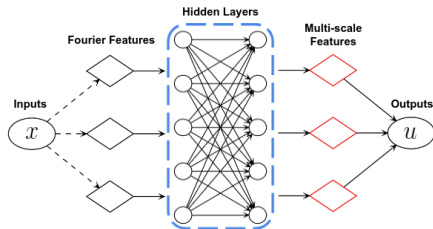
$$\gamma(x) = [\cos(Bx), \sin(Bx)]$$

Где B — матрица размера $(k \times d)$,
 k — количество признаков Фурье,
 d — размерность входа x .

Значения матрицы B обычно семплируются из нормального распределения и фиксируются:

$$\Gamma_{ij} \sim \mathcal{N}(\mu, \sigma^2)$$

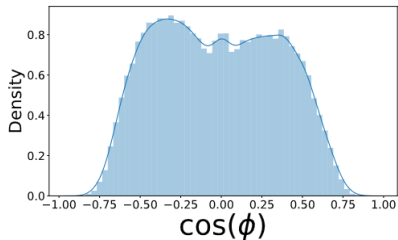
μ и σ — гиперпараметры



Wang et al.: On the eigenvector bias of Fourier feature networks: From regression to solving multi-scale PDEs with physics-informed neural networks

Конфликт градиентов

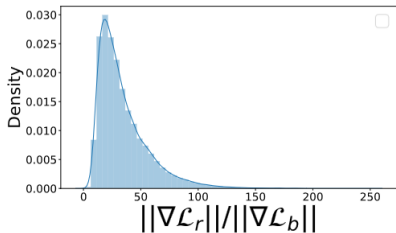
Лосс — мульти-задача: $\mathcal{L} = \mathcal{L}_r + \mathcal{L}_b$



Следствие:

При равных весах один член лосса полностью доминирует, второй фактически игнорируется.

Вывод: нужно взвешивать члены лосса.



Взвешенная целевая функция:

$$\mathcal{L}(\theta, \pi) = \sum_{i=1}^{M_r} \pi_{r,i} \mathcal{L}_{r,i}(\theta) + \sum_{j=M_r+1}^M \pi_{b,j} \mathcal{L}_{b,j}(\theta)$$

Веса π обновляются *вместе* с параметрами θ .

Learning Rate Annealing

$$\pi_i^{t+1} = \alpha \frac{\max_j |\nabla_j \mathcal{L}_r(\theta^t)|}{\frac{1}{d} \sum_{j=1}^d |\nabla_j \mathcal{L}_i(\theta^t)|} + (1 - \alpha) \pi_i^t$$

Если градиенты члена i малы — вес поднимаем.

ЕМА для устойчивости.

Saddle-point / Entropy Weighting

$$\min_{\theta} \max_{\pi} \mathcal{L}(\theta, \pi)$$

Седловая переформулировка.

Mirror descent на симплексе π .

Постановка начально-краевой задачи

Большинство практических задач — эволюционные:

$$\begin{cases} \frac{\partial u}{\partial t} + \mathcal{R}_i[u](x, t) = f_i(x, t), & i \in [1, M_r], x \in \Omega \\ \mathcal{B}_j[u](x, t) = g_j(x, t), & j \in [M_r+1, M], x \in \partial\Omega \\ u(x, 0) = u_0(x) \end{cases}$$

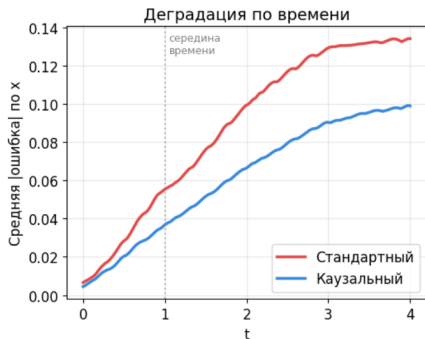
Пример: уравнение теплопроводности:

$$\frac{\partial u}{\partial t} - \nabla \cdot (a(x) \nabla u) = f(x, t)$$

Causal Training

Проблема: Ошибка растёт во времени.

Причина: в стандартном лоссе все точки (x_m, t_j) равноправны — нет мотивации учить решение по порядку времени.



Решение: взвесить лосс по времени:

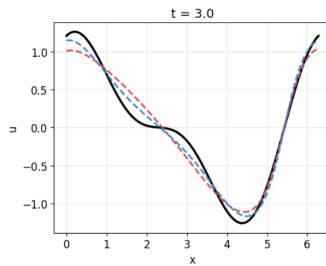
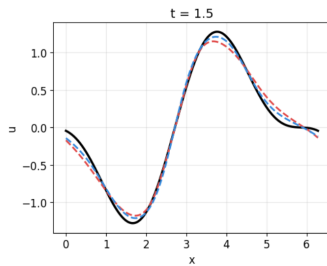
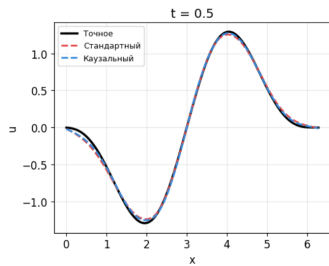
$$\mathcal{L}_{r,i}(\theta, x_m) = \frac{1}{N_t} \sum_{j=1}^{N_t} w_j \cdot \mathcal{L}_{r,i}(\theta, x_m, t_j)$$

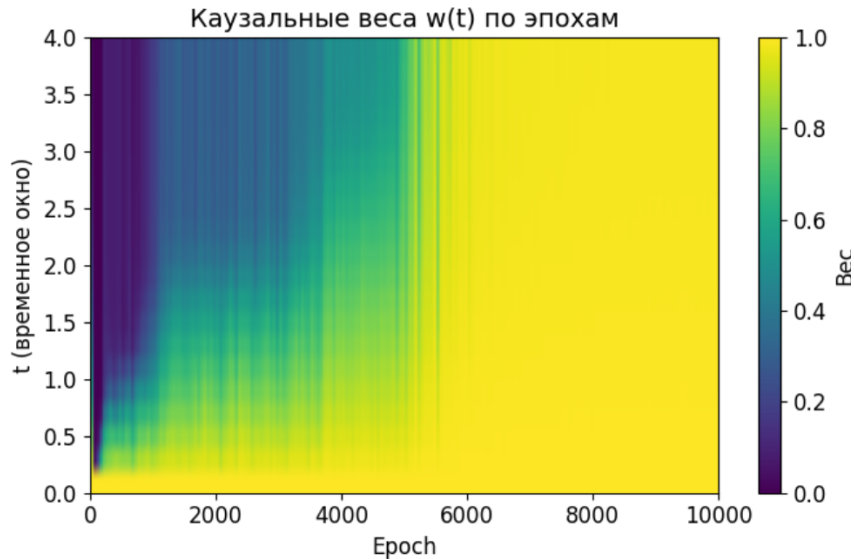
где вес $w_j = \exp\left(-\epsilon \sum_{k=1}^{j-1} \mathcal{L}_{r,i}(\theta, x_m, t_k)\right)$.

Интуиция: вес w_j маленький, пока ошибка во все предыдущие моменты не упала.

ϵ — гиперпараметр.

Сравнение



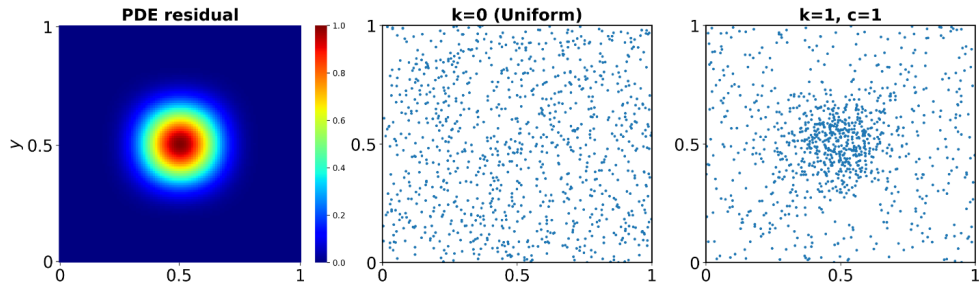


Где брать точки x_r^n ?

- **Uniform.** Базовый вариант
- **RAR.** Добавлять точки там, где большой residual
- **RAD.** Ресемплирование $\propto$ residual

RAR/RAD работают ортогонально взвешиванию: можно комбинировать.

Пример ресемплирования



PINN

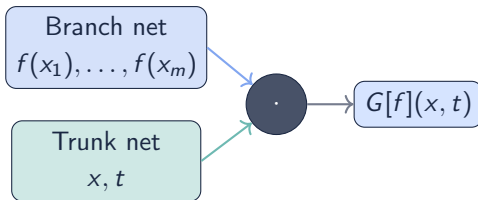
Учит **одно решение** $u(x)$
для фиксированных
параметров задачи.

Переобучение при смене
параметров.

Нейрооператор

Учит **отображение** $G : a \rightarrow u(x)$,
где a — параметры задачи.

DeepONet:



$$\mathcal{L}(\theta) = \frac{1}{N} \sum_n \sum_{a \in \mathcal{A}} \|G_\theta[a](x_n) - G[a](x_n)\|^2$$

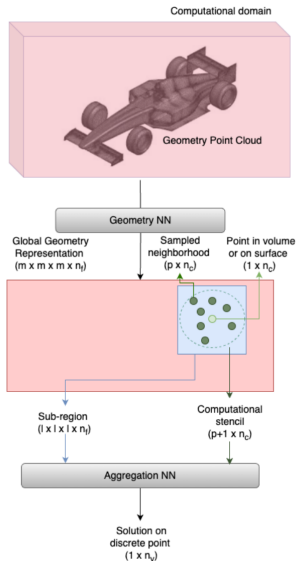
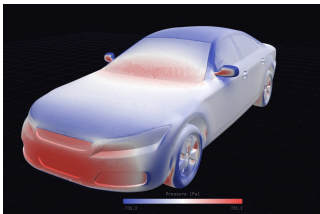
Важное отличие: для обучения нужна **выборка наблюдений** по разным конфигурациям системы.

Операторы для масштабных симуляций: DoMINO

Nvidia DoMINO — операторы для промышленных симуляций.

Пайплайн:

1. Облако точек $\rightarrow$ сетка $m \times m \times m \times f$ через свёртки: $y_i = \sum_j \kappa(x_i, x_j, d_{ij})$
2. Локальное геом. представление: свёртки + FC-слои
3. Физические параметры + геометрия $\rightarrow$ решение



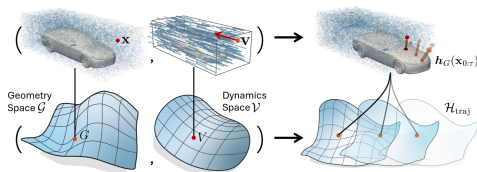
Идея: предобучить модель на
неразмеченных геометриях.

Синтетическая динамика:

$$\frac{dx^t}{dt} = v \cdot 1_{\Omega}\{x^t\}, \quad x^0 = x, \quad v \sim \mathcal{U}(\mathcal{B}(0, v_{\max}))$$

Цель — предсказывать траектории
 $\{h_{\Omega}(x^t)\}_{t=0}^T$:

$$\mathcal{L}(\theta) = \mathbb{E}_{x, \Omega, v} \|G_{\theta}(x, \Omega, v) - h_{\Omega}(x^{0:T})\|^2$$



Результаты:

- Требования к данным: –20–60%
- Скорость обучения: $\approx \times 2$

Спасибо!

Вопросы?

email: aleksandrovm@my.msu.ru

BRAIn Lab